

Abstract:

The prosperity of cloud computing has greatly expedited the developments of commercial cloud software (e.g., Airbnb, Netflix, and Robinhood) being composed by diverse micro-services (e.g., the S3 storage service and the PyTorch AI service). Compared to developing each cloud software in the traditional (monolithic) manner, the micro-service (composition) developing manner has three natural strengths in the perspective of serving enormous clients' requests. First (continuation, or reliability in this proposal), each micro-service's development party can focus on optimizing and updating its own service without interrupting the continuous deployments of other micro-services' parties. Second (performance), each cloud software can leverage the increasingly powerful accelerators (e.g., FPGA, DPU and GPU) on each computer host to greatly improve throughput or to mostly reduce the latency of specific micro-services in the software. Third (security), since each micro-service can be independently analyzed by security tools, it could be feasible to maintain a distinct and low software Trusted Computing Base (TCB) for a client request processing sensitive data, as only the micro-services involved in processing the request needs analyzing (e.g., a cryptocurrency trade request in the Robinhood software does not need to analyze the vast number of micro-services for traditional stock trades in the software).

Unfortunately, despite much academic and industrial effort, these natural strengths of micro-services still face open and significant problems in the research domains of security and reliability. For instance, bugs in the cloud privilege management software (e.g., the Linux OS kernel and the Azure software on managing hardware resources), monolithically constructed with tens of millions LoC of TCB, can easily breach controls of all micro-services running in one cloud to attackers (e.g., Azure just announced and embarrassingly fixed a severe "privilege and remote takeover" bug in Oct 2022).

Worse, each modern cloud software (e.g., Netflix) is embracing both more types and a higher number of accelerators (e.g., FPGA, DPU and GPU) in order to scale-out the performance of specific micro-services, and an accelerator has a notoriously higher probability of hardware failures than the probability of a CPU's failures (e.g., reported in the literature, the manifestation probability of failures caused by overheating and driver bugs in an accelerator can be 30X higher than the probability of those failures in a CPU). The fail-over of a specific accelerator can turn down all micro-services on the same computer host (e.g., an AWS hardware outage in 2017 turned down almost all commercial cloud software on AWS, including Netflix and Airbnb).

Although Trusted Execution Environments, or TEE (e.g., Intel SGX and ARM TrustZone), provide a promising solution to eliminate the cloud privilege management software from the TCB, both existing academic and industrial TEE-based security supporting systems are still constructed in a monolithic way. Specifically, any software security bug or hardware failure on any accelerator of one computer host can compromise or turn down an entire security supporting system and all the micro-services protected by the system, so these systems are not designed to tackle the two aforementioned security and reliability problems.

Therefore, this proposal plans to build a novel, micro-kernel inspired software architecture by pursuing three significant research objectives. Specifically, we plan to create the first micro-kernel-compatible TEE system in order to enable novel security and reliability isolation features. For instance, bugs of the privilege management software in any hardware component (e.g., bugs in the drivers of a specific accelerator or a CPU) will not affect the continuation of the micro-services that are not involved with the hardware component. We also plan to present new distributed fault tolerant algorithms and protocols with proved reliability guarantees in order to automatically and correctly recover only the stateful micro-services with only negligible performance overhead in normal deployments. Our latest preliminary results are published in international top systems conferences, including ACM Micro 2022 and Usenix ATC 2022.

The success of this proposal will promote the world-wide research and developments of micro-services and their security and reliability supporting systems. For example, the PI's (Dr. Cui's) recent academic prototype (UTEE), a secure big-data and AI supporting system, has been commercialized as the core system of the Trusted and Intelligent Cloud Services of Huawei in 2021 (huaweicloud.com/product/tics.html). In addition, many financial, AI, and entertainment enterprises in Hong Kong will be more willing to share and to deploy their original micro-services and the commercial software composed by these micro-services on public clouds.

Objectives:

1

[To develop a secure (TEE-based) micro-kernel software architecture with low-TCB, security isolation, and spatial isolation of general accelerators for micro-services]

All relevant TEE-based supporting systems that are newest and internationally notable (including [BlackBox OSDI 2022], [Occlum ASPLOS 2020] and [Graviton OSDI 2018]), which are all constructed in a monolithically way and contain an excessive TCB (e.g., including all accelerators' drivers LoC) for all micro-services running on the same computer host; in contrast, our proposed architecture in Objective 1 should maintain a near-minimal TCB for each micro-service and each cloud software (e.g., Netflix): only the micro-services and accelerators used by this software are in the software's TCB, and security bugs should not compromise the confidentiality or integrity of any other micro-services or accelerators not being used by this software. In addition, our proposed architecture should be the first work to enable each accelerator to run multiple micro-services inside, in order to maximize accelerator's computational utilization and to save energy; different micro-services running on the same accelerator does not affect confidentiality or integrity of each other (spatial isolation).

2

[To develop a secure and fault-tolerant (reliable) software architecture for automatically handling fail-over of general accelerators for micro-services]

Compared to the latest and relevant fault-tolerant (reliable) supporting systems (including [LineageStash SOSP 2019], which belongs to the replaying and recovering approach; and [Plover NSDI 2018], which belongs to the primary-backup replication approach), our proposed fault-tolerant architecture and algorithms in Objective 2 should be able to (1) achieve the best high continuation strength (as strong as the primary-backup replication approach); (2) replicate only stateful micro-services (i.e., achieve nearly the same low replication hardware costs as the replaying and recovering approach) and handle cascading (co-related) failures among accelerators and computer hosts; and (3) tackle "replaying and floating-point computation precision loss attacks" on accelerators, as well as the "targeting DoS attacks" against victim micro-services (e.g., concert ticket booking).

3

[To develop a micro-kernel supporting system for securely and automatically partitioning an AI micro-service across trusted processors/accelerators and untrusted accelerators]

To ensure a low TCB, this proposal divides all general TEE-enabled CPU processors and all general accelerators (e.g., FPGA/DPU/NPU/GPU) into two categories: (1) trusted hardware, including processors with TEE enabled and accelerators consisting of a tiny TCB (e.g., each FPGA/NPU/DPU's drivers has only up to tens of thousands LoC); (2) untrusted hardware, typically GPUs, each of whose drivers typically has millions of LoC of TCB (e.g., an Nvidia GPU's drivers and privilege software code take up about 10% of LoC in the Linux-5.8

codebase). In this proposal, the trusted hardware typically conduct non-Matrix computations (e.g., packet filtering in DPU and crypto computations in FPGA), and the trusted hardware (i.e., GPU) conducts general matrix-compatible computations (including video encoding/decoding, image rendering, audio coding and neuron computation).

Given an AI micro-service (e.g., a pre-trained deep neural network model), our proposed supporting system in Objective 3 should be fully automatic on (1) identifying the matrix-compatible computations (e.g., the convolutional layers' computations) in the micro-service; (2) partitioning these computations out of our system's TCB (i.e., CPUs and trusted accelerators) to untrusted accelerators (i.e., GPUs); and (3) conducting novel morphing algorithms proposed by us to protect the confidentiality and integrity of such computations running in the untrusted (vulnerable) accelerators.

Other Information:

Overall, when integrating all three objectives into our resultant system prototype (Objective 3c), our prototype should achieve low performance overhead: the end-to-end normal deployment execution time of micro-services in our entire system compared to the vanilla (insecure and unreliable) execution time of these micro-services should be minimized (around 10%).

Pathway to Impact statement:

Micro-services, or earliest named as “micro-web-services” in 2005 by Prof. Peter Rodgers, have nowadays become pervasive and essential in composing almost all commercial software in clouds. In 2020, a notable report ([citation1]) predicted that the global micro-service cloud market will reach USD \$3.1 billion by 2026. In 2021, Nasdaq announced to start moving its trading platforms to Amazon’s Cloud, or AWS ([citation2]). Each cloud software composed by micro-services (e.g., the PyTorch AI service or the AWS S3 storage service) has diverse natural strengths, including (1) allowing many parties to develop and to update its own micro-service independently, (2) enabling elastic performance scaling of specific micro-services using diverse accelerators, and (3) easing the secure, low-TCB (with regard to software, in this proposal) protection of processing a client request’s sensitive data on only a subset of micro-services being involved in processing the specific request.

Moreover, the emergence of accelerators (e.g., FPGA, DPU, SmartNIC, NPU, GPU, and storage-computing integrated chips) in clouds has further expedited the popularity of micro-services. Nowadays each computer host in clouds typically equips dozens of types of accelerators, with an increasingly higher number for each type of accelerator (e.g., eight GPUs per-host is getting popular) on each host. Each cloud software being composed by micro-services is particularly promising to harness the power of these accelerators, as each type of micro-service’s distinct performance requirement desires the micro-service being deployed on a specific type of accelerator (e.g., a cryptocurrency micro-service desires ultra-low latency of public key operations using FPGA, and a network-oriented micro-service desires high packet-filtering throughput using DPU).

Unfortunately, the prosperity of micro-services and accelerators in clouds also introduces significant research problems of security and reliability for the cloud software, due to two main reasons. First, the cloud privilege management software (e.g., the Linux OS and the Azure software on managing hardware resources) is still constructed in a monolithic way with tens of millions of LoC of TCB, so bugs in the privilege software can easily undermine the aforementioned security strengths of micro-services. For instance, Azure just announced and embarrassingly fixed a severe “privilege and remote takeover bug” in Oct 2022 ([citation3]). Moreover, Linux is developed by enormous parties around the world, and numerous careless or even most malicious bugs in the Linux OS kernel have been announced every year.

The second main reason is that, drivers and privilege software of each type of accelerators is developed by enormous parties; all the accelerators on a computer host are also used and managed by the cloud privilege management software in a monolithic way. This proposal aims to support general accelerators, and we divide all general TEE-enabled CPUs and all general accelerators into two categories: (1) trusted hardware, including processors with TEE enabled and accelerators consisting of a tiny TCB (e.g., each FPGA, NPU, or DPU’s drivers has only up to tens of thousands LoC); (2) untrusted hardware, typically GPUs, each of whose drivers typically has millions of LoC of TCB (e.g., an Nvidia GPU’s drivers and privilege software code take up about 10% of LoC in the Linux-5.8 code base). We consider this dividence of TCB reasonable, because the TCB of a GPU’s (second category) drivers and privilege software is not only developed by many diverse (naturally, mutually untrusted) parties, but this TCB in a GPU is also as huge as that of a Linux OS kernel; in contrast, our first category’s drivers and privilege software are often developed by few parties and the LoC is both small and easy to be systematically tested by security tools ([citation4,citation6]).

Another part of the second main reason is that, it is fundamentally difficult to pause and then to resume a GPU’s ongoing computation (as it contains tens of thousands of Matrix-compatible processors) efficiently. However, pausing, updating, and then resuming (or “hot-patching” in academia) the drivers of our first category (e.g., FPGA, NPU, and DPU) are readily feasible with existing efficient solutions. Worse, the OS and privilege cloud management software of each computer host are still invoking and managing accelerators in a monolithic way: all accelerators are loaded into the same memory address space for each cloud software (e.g., one space for the entire Netflix and another space for the entire Robinhood), and the OS kernel itself is a monolithic space. Therefore, a software security bug or hardware failure (e.g., overheating) on any accelerator can compromise or turn down all software and micro-services on the same computer host, or even many hosts in the cloud.

Since these security and reliability research problems are often mingled, this proposal will tackle them at once. In recent years, TEE (e.g., Intel SGX and ARM TrustZone) techniques in CPUs have been actively developed by academia and industries in order to eliminate the cloud privilege management software from the huge TCB. Since TEE can have much simpler (smaller) logic than the privilege management software, and TEE products can be completely tested by tools (e.g., EDA tools), TEE is viewed as trustworthy and has been used to protect extremely sensitive “small data” (e.g., fingerprints and passwords). However, despite much effort from worldwide academia and industries, all existing TEE-based security supporting systems (including [BlackBox OSDI 2022], [Occlum ASPLOS 2020] and [Graviton OSDI 2018]) are still designed in a monolithic way, so a software security bug or hardware failure in an accelerator can compromise or turn down the entire supporting system and micro-services on this host. Therefore, existing systems are not designed to tackle the aforementioned security and reliability problems, which will be tackled in this proposal.

This proposal presents a novel, micro-kernel inspired software architecture by pursuing three significant research objectives, effectively eliminating the “monolithic” limitations in these existing systems. Specifically, we plan to create the first micro-kernel-compatible TEE system in order to enable novel security and reliability isolation features. For instance, bugs of the privilege management software in any hardware component (e.g., bugs in the drivers of a specific accelerator) will not affect the continuation of other micro-services and other accelerators that are not involved with the hardware component.

In the short term (next 1 to 4 years), in order to gain both academic and industrial impacts, we plan to publish the results and system prototype of our objectives in international top software systems conferences and journals (e.g., SOSP, Micro, ATC, TPDS, and TDSC). Our preliminary results (proposed in **Objective 1 and 3** in this proposal) are published in international top systems conferences, including [UTEE AsiaCCS 2020], [Cronus Micro 2022] and [Soter ATC 2022]. A subset of our preliminary system prototype (proposed in **Objective 1**, UTEE and Cronus) has been commercialized as the core system of Huawei Cloud’s Trusted and Intelligent Cloud Services (TICS-1.0) launched in 2021 (huaweicloud.com/product/tics.html). Huawei’s official acknowledgment letter (hemingcui.github.io/doc/utee-ack.pdf) confirmed that, Dr. Cui’s (the PI’s) preliminary system prototype (UTEE and Cronus) is already usable by Huawei Cloud’s 3 million customers (including enterprises and individuals) in 170 countries.

In the medium term (next 5 to 9 years), we envision that our proposal will lead to broader impacts in the Computer Science discipline. The deliverables in this proposal will make themselves the most competitive candidates for future TICS releases (e.g., TICS-2.0). Since the deployments of general micro-services on general accelerators will be much more secure and reliable, we envision that more and more enterprises will be willing to share and to deploy their originally-invented micro-services on clouds. We will collaborate with broad industries, including but not limited to Huawei, NEC, and AWS. For instance, the PI sent his PhD student Jianyu Jiang to conduct a collaboration with the NEC Lab in Germany, and the student published a paper with this NEC Lab’s researchers about TEE security in the RAID (Research in Attacks, Intrusions and Defenses) conference in 2022.

Following the micro-kernel inspirations in our proposal, maybe worldwide computer architecture researchers will propose new CPU and accelerator architectures with specific designs for securing general micro-services. Computer verification researchers may develop novel, micro-kernel-oriented theories and tools to design and verify new TEE products (new CPUs), making TEE more pervasively used in diverse scenarios, including securing, auditing, and governing public clouds. Maybe some future-generation CPUs and accelerators will also be designed in a “micro-kernel inspired way” (as this proposal) in order to match the future deployment requirements of micro-kernel OS and micro-services on clouds.

In the long term (10 years or later), we envision that the deliverables and impacts of this proposal will go beyond Computer Science and lead to innovations in broader disciplines, including finance and business (e.g., Robinhood and Nasdaq), social media and entertainment (e.g., Slalom, TrackIt, Amdocs, LTN, Amagi, Netflix, Ant Media, and Bluescape, see the list of the industries in [citation5]), and physical sciences (e.g., the USA NASA). The resultant software systems in our proposal may promote more frequent data sharing securely across different disciplines on clouds and lead to more collaborations. Hong Kong financial industries may compose more intelligent and profitable software easily with micro-services and our proposed micro-kernel systems; more startups will also compose software in a “micro-” (low-TCB, low-cost, and flexible) way using our micro-kernel supporting systems. Social media and entertainment creators may leverage the prototype in our proposal to create and to share more entertainment data securely on clouds. Last but not least, world-wide physical scientists, organizations, and governments may initiate more collaborative scientific experiments and research plans on clouds, with the greatly enhanced security and reliability enabled by our proposal.

Citations used in the Pathway to Impact statement:

[citation1] Research, Verified Market. Instant Tech Market News, 2020-02-18. "Cloud Microservices Market 2020 Trends, Market Share, Industry Size, Opportunities, Analysis and Forecast by 2026"

[citation2] Nasdaq to Move Markets to Amazon’s Cloud. The Wall Street Journal, 2021-11-30. <https://www.wsj.com/articles/nasdaq-to-move-markets-to-amazons-cloud-11638291103>

[citation3] Azure vulnerability opens door to remote takeover attacks. [www.techtarget.com](https://www.techtarget.com/searchsecurity/news/252526281/Azure-vulnerability-opens-door-to-remote-takeover-attacks), 2022-10-19. <https://www.techtarget.com/searchsecurity/news/252526281/Azure-vulnerability-opens-door-to-remote-takeover-attacks>

[citation4] M. Renzelmann, et. al. SymDrive: Testing Drivers without Devices. OSDI 2012. <https://github.com/riyadparvez/symdrive>.

[citation5] Media and Entertainment Partners on AWS. <https://aws.amazon.com/cn/media/partners/>

[citation6] A. Machiry, et. al. DR. CHECKER: A Soundy Analysis for Linux Kernel Drivers. Usenix Security 2018. https://github.com/ucsb-seclab/dr_checker.

1 Research Context

1.1 Secure, Reliable and High-performance Micro-services

The emerging micro-service paradigm [1–7] enables users to run diverse web applications (e.g., WeChat [8], Social Network [1], E-commerce [9], SmartCity [10], Clipper [11], Tensorflow Serving [12] and TensorRT [13]) with high reliability and high performance. The key insight is to compose a sophisticated (monolithic) application using decoupled and simple micro-services (e.g., Memcached [14] and Storm [15]), where micro-services communicate through remote procedure call [16–18] (RPC). The micro-service paradigm has been adopted in many public clouds (e.g., AWS [19] and Azure [20]) and world-leading technology companies (e.g., Uber [21], LinkedIn [22] and Netflix [23]) for serving billions of people’s requests (e.g., face and social data).

Compared to traditional monolithic services, this proposal summarizes four unique features for micro-services, by conducting extensive studies on existing micro-services in the academia (e.g., DeathStarBench [9], μ Suite [24], Trainticket [25], OpenWhisk [26], Camelot-Suite [27] and Microbenchmarks [28]) and industry (e.g., AWS S3 [29], Lambda [30], Fargate [31] and Azure Service Fabric [32]).

- **M1: Multi-party development.** Each micro-service is usually developed by a single party and is isolated from other micro-services [1, 22, 33, 34]. For example, Tensorflow Serving [12] and Fargate [31] are developed by Google and Amazon, respectively. This is beneficial as each micro-service is maturely developed, optimized and maintained. A cloud tenant can develop an software (e.g., E-Commerce [9]) simply by integrating and executing micro-services in a directed graph manner [1], without the tremendous efforts in optimizing each micro-service. In public clouds, thousands of micro-services are available for building applications for millions of applications [1].

- **M2: Short-live execution.** Micro-services are usually short-live to maximize utilization of hardware resources. Specifically, a micro-service’s request execution is short (typically within seconds, see Fig. 1), and the request is temporally [1] or spatially sparse [35]. Hence, public clouds typically runs hundreds of micro-services in the same machine and share the same hardware (e.g., accelerators) temporally and spatially [5, 7, 36, 37]. Yet, doing so requires fast startup and termination of micro-services, as well as low switching time between micro-services [18].

For example, since a micro-service’s execution time is usually within seconds, a micro-service’s startup time is optimized within 100ms in AWS [18, 36] and even within sub-millisecond in many research systems [5, 7].

- **M3: Diverse accelerator usage.** Each kind of micro-service typically make use of a specific accelerator (e.g., GPU, FPGA, and TPU) to achieve holistic performance gains [38, 39]. For example, AWS EMR [40] uses GPU and FPGA for accelerating data analytics with 2X ~ 10X throughput; while Azure Cognitive Services [20] leverages GPU and NPU for DNN inference with extremely low latency; AWS is collaborating with diverse hardware vendors such as Intel and Xilinx to accelerate storage services using FPGA for a up to 10X throughput [41, 42]. Hence, a cloud machine is integrating more and more accelerators for meeting the performance requirements of different micro-services [43, 44]. Moreover, although many accelerators (e.g., FPGA) enables ease-of-use hardware updates (e.g., FPGA reconfiguration [42]) for fixing security or performance bugs in the accelerators [45, 46], many matrix-compatible accelerators (e.g., GPU) are too sophisticated in hardware (w/ video playback, graphics and tensor computing) and software (e.g., NVIDIA’s 934k LoC GPU driver [47]), unfavorable to be trusted [48].

- **M4: Sensitive data processing.** The simplicity of micro-services (e.g., Memcached [14] and Storm [15]) makes it extremely suitable for processing a large amount of sensitive data in the clouds [12]. Specifically, by integrating and executing micro-services in a directed graph manner, an application can contain a flexible and sophisticated data processing workflow (e.g., machine learning pipeline [49]) with high performance [1, 12, 50] and few developing efforts.

Due to the increasing threats against clouds [32, 51, 52], Trusted Execution Environment [53–59] is a natural fit for securing micro-service execution. TEE typically provides an isolated execution environment called enclave, which cannot be seen or tampered with by malicious attackers. Due to the high security insurance of TEE, we consider TEE as a key ingredient for securing micro-services in clouds.

To guide the design of all three objectives, we conducted extensive literature study and distill the four features into four technical goals when securing micro-services within TEE to process sensitive user data.

- **O1: Security.** The security of micro-services is in the first priority due to the processing sensitive data in clouds (M4) and the blooming of data crimes and leaks [60–62]. However, although each micro-service itself typically maintain a small TCB, micro-services are all composed monolithically (e.g., Linux contains 1M LoC) with high security risks. This is aggravated by the integration of accelerators (M3). For example, a NVIDIA GPU contain sophisticated hardware and up to 1M LoC of its privileged software (see M3), while an accelerator may be injected with malicious code to compromise all other accelerators running in the same machine [48, 63]. Moreover, ensuring a correct execution (e.g., exactly-once execution [3]) of the distributed micro-services (M1) are also known to be challenging. Overall, micro-services demands low-TCB, security-isolated and correct execution.

- **O2: Reliability.** The reliability of a micro-service is extremely important, as a single micro-service’s failure can cause cascading failures of all its dependent micro-services due to the directed graph execution of micro-

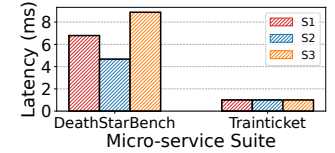


Fig. 1: Execution time of two notable micro-services suites.

services (M1), potentially incurring billions of dollars of losses (e.g., Amazon S3 outage in 2017 [29]). Worse, long-term analysis [64, 65] on server failures shows that accelerators are susceptible to generating 10X more faults than CPU-only machines, making accelerator-enabled micro-services (M3) significantly easier to fail. The two difficulties have been aggravated by the notorious “replaying and floating-point computation precision loss attacks” attacks [66] and targeted DoS attacks [67], which targets failure recovery to induce incorrect results for security critical software (e.g., navigation) or delay stringent requests. It is thus extremely crucial to ensure *non-stop* execution (continuation) of micro-services upon failures by recovering micro-service data correctly and securely.

► **O3: High performance.** While achieving the security and reliability objects (O1 and O2), it is crucial to maintain low-overhead and low-cost micro-service execution. (1) Due to the short-live characteristic (M2) and high performance overhead of RPCs in TEE [37, 62, 68], applying traditional monolithic TEE runtime (e.g., Graphene-SGX [69] and SCONE [70]) for micro-services results in prohibited performance overhead (e.g., 2X computing latency and starting up with seconds). (2) It is important to enable temporal and spatial sharing of hardware resources, in order to maximize hardware utilization (minimizing costs). This is extremely crucial, as accelerators (e.g., FPGA and GPU) are typically more expensive and precious than CPU resources. For example, an A100 GPU costs USD 30,000, which is equivalent or even more expensive than a whole high-end CPU-only server. However, due to the short-live characteristic (M2), preserving a dedicated accelerator for a single micro-service (tenant) often results in extremely low utilization (e.g., 10%) on every accelerator [71, 72].

This proposal focuses on protecting sensitive user data to accommodate the three heterogeneity characteristics for secure, reliable and high-performance micro-service execution. This proposal first asks a fundamental research question (**RQ I**): **How should an OS be designed to accommodate micro-service workloads within TEE?**

Obj. 1 answers the question with a key insight: when a monolithic service is partitioned into micro-services, the OS should also be partitioned into MicroOS. To this end, **Obj. 1** designs a security-isolated and high-performance OS architecture, MicroTEE, for executing micro-services using general, trusted accelerators, while maximizing the accelerators’ resource utilization (meeting O1 ~ O3). The MicroTEE architecture comes with an Streaming Remote Procedure Call (sRPC) protocol to support secure and efficient RPCs between micro-services and MicroOSes, and a zero-copy CPU-accelerator transmission protocol for fast CPU-accelerator communication.

Further, to facilitate the reliability goal of the proposed design (O2), **Obj. 2** focuses on developing protocols and abstractions for fault-tolerant, highly available micro-service execution. The objective answers a key research question (**RQ II**): **How to securely and efficiently ensure reliable micro-service execution within TEE?** **Obj. 2** tackles the reliability goal (see §1.1) in micro-services by leveraging the insight that micro-services typically executes in a synchronous/asynchronous manner, and faults merely happen. Then, it is possible to handle faults asynchronously without stalling the execution. Leveraging this insight, **Obj. 2** creates replication and failover protocols to ensure correct, non-stop micro-service execution with Service-level Agreements (SLA) guarantee.

Obj. 1 and **Obj. 2** focuses on enabling trusted execution of a single micro-service within TEE, yet a micro-service can often be further partitioned and be accelerated by untrusted (matrix-compatible) accelerators such as GPUs (M3). **Obj. 3** aims to tackle an important research question (**RQ III**): **How should a micro-service be efficiently and securely partitioned and executed?** To this end, **Obj. 3** aims to develop algorithms for further partitioning a micro-service into trusted and untrusted parts, and offloads the untrusted parts to untrusted accelerators. Then, **Obj. 3** carries protocols for ensuring integrity (correctness) of the directed graph execution of micro-services (M1). Overall, by unifying all the objectives above and integrating the inter-micro-service integrity protocol, **Obj. 3** ensures execution of commodity, off-the-shelf micro-services with O1/O2/O3.

1.2 Related work by others

TEE-based systems. Existing work either adopts the shielding approach [69, 70, 73, 74] to run unmodified applications within TEE enclaves, or customize applications using the TEE’s SDK [61, 75–80]. By doing so, TEE has been increasingly used for protecting the confidentiality and integrity of many machine learning (ML) systems such as data analytic [76, 77], streaming data processing [81, 82], and DNN training and inference [83–89]. Recent work tries to protect micro-services within TEE by creating hardware primitives for lightweight startup [37] and designing software protocols for service accounting [90]. However, despite tremendous efforts, all existing work composes micro-services monolithically, which can hardly satisfy our three goals (O1/O2/O3).

Preserving integrity of distributed micro-service executions. Many research work [61, 76, 77, 91–93] ensures the correctness of the computed results by creating protocols for distributed enclaves executed in specific manners (e.g., directed acyclic graph), or by ensuring lock-step execution for general enclaves [78, 94]. However, no existing work targets micro-service executed in a directed graph manner (M1). **Obj. 1b**, **Obj. 3b** and **Obj. 3c** aims to guarantee the integrity (correctness) of micro-service executions.

Fault tolerance, High availability and Service-Level-Agreement (SLA) of Micro-service. There exist many research work on providing fault tolerance [33, 95], high availability [8, 96–98] and SLA [50, 99] for micro-services. Despite these brilliant work, existing work assumes a trusted, reliable (monolithic) OS kernel, which is not realistic in practice and cannot defense against cloud attackers [53, 54]. Worse, they cannot ensure fast recovery from failures incurred by accelerators [66, 100, 101].

1.3 Related work by the PI's team

The PI's team is experted in building secure, reliable and high-performance distributed systems with active publications in recent years, including secure TEE systems [61, 62, 102–106], privacy-preserving distributed big-data systems [61, 102, 103, 107, 108], and static program analysis [109]. Relevant to this proposal, the PI received a best paper award in ACSAC 2017 on building the Kakute secure data analytic [108]. The PI's team also collaborated with Huawei to commercialize a secure TEE runtime for big-data (Uranus [61]) as Huawei Cloud's key secure TEE runtime [110] and collaborated with NEC Laboratories Europe for analyzing TEE security (initial results published at RAID 2022 [106]), which guides the design of our three objectives. As preliminary results of **Obj. 1a**, the PI published a novel TEE OS architecture for accelerators in Micro 2022 [62], and the PI's team and China Telecom have reached initial agreements for developing and deploying the new TEE OS in their public clouds; as preliminary results of **Obj. 2a**, the PI published a high availability workflow paper in DSN 2020 [66] for non-stop inference micro-services; as preliminary results of **Obj. 3a**, the PI has published an algorithm for partition micro-services on DNN inference in ATC 2022 [104]. Due to these preliminary results and impact, Huawei conferred a "best collaborating scientist" award to the PI in 2021.

2 Research question and methods

We begin by introducing our threat model and failure model. §2.1 ~ §2.3 introduces **Obj. 1 ~ Obj. 3**.

Threat model. The proposal has a delicate threat model optimized for micro-services. For an application run by a tenant, the tenant trusts only the used (application-composing) micro-services running within TEE, including the micro-service software, their runtime (e.g., the RPC library), micro-services's dependent accelerator driver (and firmware) and accelerator hardware, and TEE hardware. Other (thousands of) unused micro-services, including the micro-service software and their runtime, and their dependent (tens of) accelerators' hardware and (privileged) software, as well as the used micro-services' software and hardware outside TEE, are not trusted and can arbitrarily monitor and tamper with the system. For example, a malicious micro-service can contain malicious software code and tries inject malicious code into other micro-services running in the same machine. The attacker may also tamper with, delay or drop the RPC messages among micro-services. This proposal's threat model is reasonable, as a cloud tenant typically conducts security audit on its used micro-services, our threat model ease the audition by maintaining a small and security-isolated TCB. Same as previous TEE-based systems [54–56, 59, 111], this proposal assumes that all trusted software and hardware are correct and do not contain any bugs for exploits (e.g., buffer overflow [109] and iago attack [112]). The proposal targets at attacks that directly leaks application secrets, so side-channels attacks [113, 114], denial of service attacks [67] and hardware attacks (e.g., bus snooping [115]) are out of the scope of this proposal and can be tackled by orthogonal work [92, 116–118].

Failure model. Similar to previous TEE systems [68, 119], this proposal assumes that all trusted hardware and software components are correct, but these components may fail due to hardware errors [64, 65], environmental issues (e.g., overheating) or a malicious attacks (by the malicious cloud managing software). Specifically, this proposal assumes crash-safety [120] of these components, i.e., the security and correctness properties of these components are still maintained when these components are crashed. Same as previous TEE systems [68, 119], this proposal does not consider byzantine faults (e.g., bit flips in accelerators [65]) in the trusted components. Nevertheless, this proposal considers that the unused micro-services and the used micro-services' components running outside TEE, may be malicious and fail arbitrarily.

2.1 A secure OS architecture with low-TCB, security isolation, and spatial isolation for micro-services.

This objective aims to answer **RQ I** with a microkernel-inspired OS. The main insight of the objective is that, when a monolithic service is becoming micro-services, the monolithic OS should also be micro-OS. To do so, we analyze existing systems and distill three technical requirements for a secure OS architecture for micro-services. First, the OS should support general accelerators without hardware customization to accommodate the diverse micro-service workload. This is because hardware customization is difficult for verifying correctness [48] and is not generic to accommodate new accelerators. Second, the OS should enable spatial sharing on accelerators to maximize resource utilization [71]. Third, the OS should enforce strong isolation between different accelerators' hardware and software stack, as accelerators may be designed and built by different vendors.

Unfortunately, despite tremendous efforts for integrating accelerators into TEE, none of the existing approaches (classified as *hardware-based* and *software-based* approach) meets all three requirements. Specifically, the hardware-based approach (e.g., CURE [56] and Graviton [121]) customizes I/O bus or hardware accelerators to enforce isolation among enclaves, yet doing so requires extensive TEE [68] or accelerator specific [121] hardware customizations. On the other hand, the software-based approach typically leverages the unmodified hardware and customizes only the software runtime [54] to form a monolithic design, which runs all accelerator software in the same address space without isolation.

Obj. 1a: MicroTEE OS architecture, a novel OS for enabling trusted micro-service execution on general accelerators and enforcing R1/R2/R3. The MicroTEE OS architecture is inspired by the microkernel paradigm [122–124]. MicroTEE partitions a monolithic enclave running heterogeneous computing tasks on different hardware (e.g., CPU-GPU coordinated DNN training) into several micro enclaves (mEnclaves) and parti-

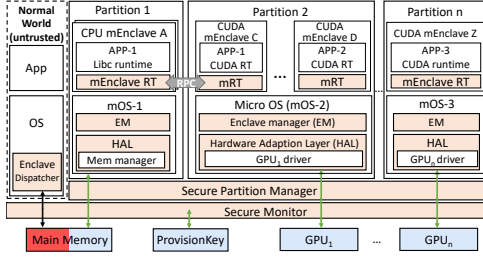


Fig. 2: Preliminary architecture of MicroTEE.

tions the monolithic OS into several micro OSes (mOSes). Specifically, each MicroEnclave encapsulates one type of computation (e.g., GPU computation) of the heterogeneous computing task, and each mOS is a shim trusted runtime that manages only one accelerator; each mOS includes only enclave and hardware management logic (e.g., drivers) for sharing hardware resources among mEnclaves. MicroTEE constructs a heterogeneous computing task using remote procedure calls (RPCs) between mEnclaves.

In order to strongly isolate each mOS, MicroTEE contains two aspects. First, MicroTEE isolates different mOSes (and its managing accelerator) by leveraging existing hardware isolation techniques (e.g., ARMS-EL2 [125]). Second, MicroTEE eliminates a core OS kernel (used in microkernels) that coordinates different mOSes by replicating basic OS functionalities (e.g., RPC) in all mOSes; MicroTEE offloads OS coordinations (e.g., enclave scheduling) out of the TEE.

Obj. 1b: sRPC (Streaming Remote Procedure Call), RPC protocols and workflows for transparently running diverse micro-services within TEE with high-performance. One main challenge arises when implementing the MicroTEE architecture. Specifically, frequent RPCs between mEnclaves can significantly downgrade performance during execution and violate **R3** during failure. Specifically, RPC usually requires context switches to make the remote mEnclave active, and recent work [111] shows that context switches within TEE can result in prohibited performance overhead. Worse, an attacker can actively reorder and replay RPCs to compromise the integrity of heterogeneous computing [68, 78] (§2) or maliciously generate failures to an mOS/mEnclave to substitute a trusted mOS/mEnclave to a malicious one, gaining sensitive user data sent through RPC (§2).

Cronus tackles the challenge with a new *streaming remote procedure call (sRPC)* protocol that mitigates frequent context switches and does RPC over shared trusted TEE memory to resist reorder and replay attacks. To tackle malicious attacks during failures, sRPC carries a *proceed-trap* failure recovery procedure that proceeds execution without downtime upon a failure and traps for access with potential data leakages to prevent leakages through failed mOSes/mEnclaves [126]. Cronus’s implementation includes a special attestation protocol for users attesting TEE with different accelerators and enables direct integration of off-the-shelf accelerator drivers with a shim mOS library that provides basic OS functions (e.g., memory mapping).

Obj. 1c: Direct Transmission Path (DTP), a new data abstraction for low-overhead data transmission between a micro-service application and a spatially shared accelerator. When data is transferred between enclaves and the accelerator, the data must be transferred to the (from the enclave) to the OS first. To achieve this, the OS needs to conduct excessive amount of sanity checks and data copies, significantly hindering the benefits of offloading computation to accelerators [127]. To tackle this problem, we leverage the observation that two micro-services will not access the same data region at the same time [2], as they typically share data using RPC. Hence, DTP makes the OS maintain a simple ownership-based isolation [122, 128] on a spatially shared accelerator’s resources among mEnclaves: an mEnclave can directly transfer the its data ownership to the mOS, and to the accelerator hardware, without copying the data. Specifically, DTP enforces the isolation by statically checking the ownership of resources at the compiled time (e.g., using a Rust-like ownership checker), and by dynamically checking the ownership using software/hardware based capability isolation [122, 128]. By doing so, the excessive amount of data copies and sanity checks are eliminated.

Preliminary result. We implemented a prototype of Cronus using ARM TrustZone. We evaluated Cronus on CPU and a VTA NPU [129]. We evaluated Cronus on the notable microbenchmark VTA-BENCH [130] for NPU, and also evaluated Cronus on real-world AI Serving micro-services using TVM [129] on NPU [129]. We evaluate the end-to-end latency of AI Serving on three state-of-art DNNs (i.e., ResNet18 [131], ResNet50 [131] and Yolo [132]) on three popular datasets (i.e., MNIST [133], Cifar [134] and ImageNet [135]), respectively. Our preliminary result (Fig. 4) shows that the performance overhead of Cronus is less than 7%, and Cronus achieves security isolation and fault isolation.

2.2 A secure fault-tolerant and reliable architecture for micro-services.

Same as micro-kernels, micro-service need to guarantee the reliability of micro-service (**O2**). Luckily, realizing the reliability of micro-services is simplified by the isolation the each MicroOS and micro-service [122]. Specifically, we answer **RQ II** by summarizing three technical requirements for micro-service reliability.

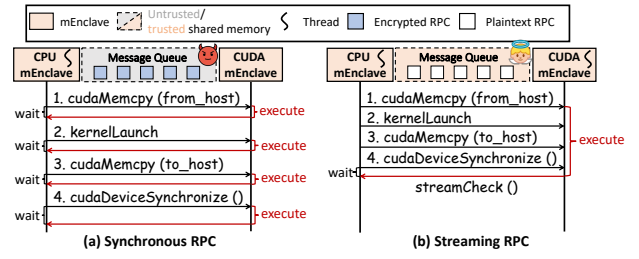


Fig. 3: The sRPC protocol.

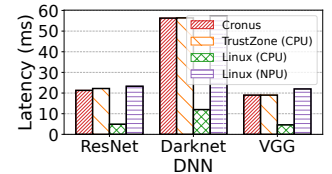


Fig. 4: Preliminary results of Obj. 1 on AI serving software.

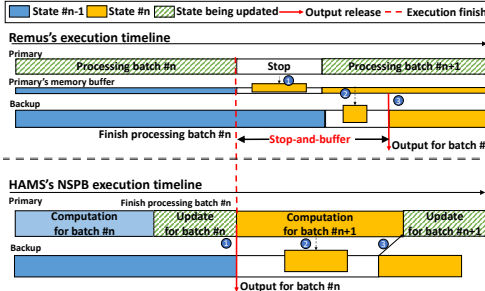


Fig. 5: The NSPB protocol.

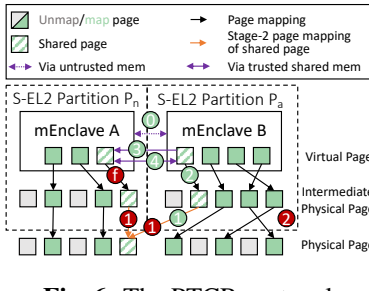


Fig. 6: The PTCR protocol.

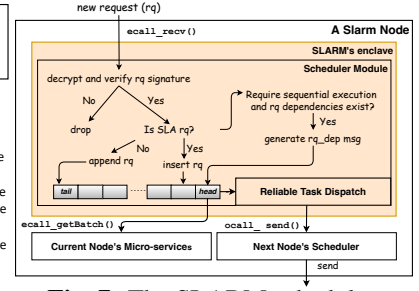


Fig. 7: The SLARM scheduler.

(1) Due to the tremendous faults incurred in accelerators (M3), it is crucial to enable *fault isolation* [122, 124] when sharing hardware resources (M2): any faults of the software and hardware stack in a micro-service should be isolated from other micro-service. By dosing so, the number of faults for each micro-service is greatly mitigated. (2) Upon failures, to accommodate the short-live execution of micro-service (M2), application data should be recovered from faults rapidly to ensure *non-stop execution*. Moreover, the recovered data should be correct (e.g., consistent and up-to-date) among the micro-services. This is extremely challenging as some micro-services in the directed graph execution [3, 33, 95] (M1) are stateful, and recovering correct data may result in cascading failures/recovery of all micro-services [66, 98]. (3) Due to sharing of hardware resources among different tenants (M3), it is crucial to schedule micro-services for meeting their Service-Level-Agreements (SLA) [50]. However, doing so is challenging due to the directed graph execution (M1) and the untrusted OS scheduler (§2), which can conduct targeting DoS attacks [67] of a specific micro-service (e.g., concert booking service)

Obj. 2a: PTCR (Proceed-trap Checkpoint/Recovery), proactive and secure live update protocols for MicroOS components. The MicroTEE architecture (§2.1) enforces strong isolation among MicroOS. However, when there are faults, the recovery process requires coordination with other MicroOSes, as there are shared data among MicroOSes. With the help of MicroTEE’s, PTCR can leverage existing checkpoint techniques (e.g., CRIU [136]) to checkpoint mEnclaves and mOSes for recovering user data during failures or task migration.

We assume an mEnclave mE_A in P_n (e.g., managing a CPU) is communicating with an mEnclave mE_B in P_a (e.g., managing a NPU), as shown in Fig. 6. A partition P_i runs an mOS O_i on a device D_i^c with configuration c (including device content). We use $smem(mE_i)$ for the shared memory in mE_i and $pt(mE_i)$ for the corresponding page table entries. We use $smem(P_i, P_j)$ for the shared memory that P_i shares with P_j , and $pt^2(P_i, P_j)$ for the corresponding stage-2 page table entries. The SMMU page table entries that P_i shared with P_j are denoted as $spt^2(P_i, P_j)$.

The PTCR protocol (Red steps in Fig. 6) recovers a failed P_a in three steps: ① PTCR first finds all partitions that contain shared memory with P_a . For a P_i of them, PTCR invalidates stage-2 page table entries (orange arrows in Fig. 6) in $pt^2(P_i, P_a)$ and SMMU entries in $spt^2(P_i, P_a)$, so that access to the shared memory is invalid. After that, all subsequent shared memory access will generate faults, removing the TOCTOU attacks. After that, PTCR’s secure monitor marks the partition P_a as failed by setting $r_f = 1$, so that all consecutive new memory sharing requests are blocked. Afterward (②), PTCR loads P_a ’s failure handling function to clear D_i^c and $smem$. After that, PTCR can load an mOS into P_a and recover the checkpoint states. After the recovery finishes, PTCR sets P_a ready by setting $r_f = 0$. PTCR recovers mEnclaves’ state checkpointed using CRIU [136].

Obj. 2b: Non-stop Primary Backup (NSPB), a novel replication protocol for non-stop micro-service execution. Although Obj. 2a enables failure recovery of mEnclaves and mOSes, the downtime of the failed micro-services is still too long. For example, one second in “Double eleven” (a Chinese shopping festival) can generate 583k TxN (\$10M)/s. Hence, a micro-service failure could potentially cause a catastrophic loss [1, 29].

Traditionally, the primary-backup approach [96] can provide non-stop program execution by creating two servers (primary and backup server) that stores identical states. Normally, the primary server serves user requests; when the primary fails, the backup server takes over immediately without downtime. However, this primary-backup approach [96] needs to stop the primary to retrieve its internal state. For instance, Remus [96] leverages VM-specific techniques (e.g., shadow page tables) to record updated memory pages (a.k.a, dirty pages) on the primary VM and periodically stops the primary VM for retrieving the dirty pages to send to the backup. However, this black-box approach is not suitable for replicating stateful micro-services executed as a service graph [1] with accelerators, as retrieving dirty accelerator memory is challenging [100, 101] and slow [66] in practice.

Unlike Remus [96], NSPB introduces a white-box mechanism based on the compute-then-update nature of micro-services [50] to retrieve a stateful micro-service’s internal state without stopping it. As illustrated in Fig. 5, during the computation stage for the $N+1$ batch, the primary micro-service’s state (i.e., parameters) stays intact, same as after processing the N batch. Thus, the primary’s proxy can retrieve the micro-service’s internal states in parallel with the computation stage of the $N+1$ batch, without the need to stop it. Occasionally, state retrieval may take longer time than the computation stage. To prevent the proxy from getting corrupted state, Hams lets the primary micro-service wait for a signal from the proxy before entering the update stage, the proxy sends the signal after finishing its state retrieval.

Obj. 2c: fine-grained Service Level Agreements (SLA) Abstraction and Scheduling Algorithms for

satisfying SLA of micro-services. Due to sharing of compute resources across micro-services, the contention of shared resources can cause latency spikes that violate the Service Level Agreements (SLA) of user requests [50, 99]. However, existing work [50, 99] for micro-service SLA cannot defend against a malicious cloud who can arbitrarily manipulate the scheduling of micro-services (running outside TEE) to sabotage the SLA of specific user requests [69, 137].

To provide SLA against malicious scheduler, this objective aims to design an oblivious scheduling protocol that guarantee SLAs of user requests. The main idea is to shuffle a large amount of user requests, making it difficult for the attacker to conduct targeted attacks against specific users. By further injecting dummy requests, denial of services (DoS) attacks that stops all attacks can be detected.

Fig. 7 shows the architecture of the SLA scheduler running inside TEE. For each request, a deadline is carried across micro-services, and the scheduler subtracts the deadline of this request with two trustworthy (conservative) time variables: `rttsymbol`, the recent worst-case elapsed time (i.e., round trip times plus execution time) between micro-service N and N' (N 's consecutive micro-service); and `waitsymbol`, the time cost a request spent on micro-service N 's SLARM enclave. SLARM makes these two variables **conservative**: the subtracted elapsed time from a request's remaining deadline is larger than the actual elapsed time on the request's execution path. Then, the scheduler hides the deadline of a request by conducting a oblivious shuffling of a bunch of requests. SLARM also integrates a resource profiler [50] for estimating the resource usage of each micro-service, in order to conduct task scheduling and migration (see §2.2) for maximizing SLA satisfaction rate. By doing so, the SLA of user requests can be guaranteed.

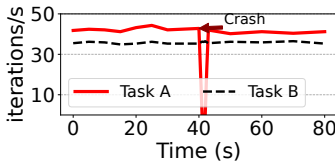


Fig. 8: mOS failover.

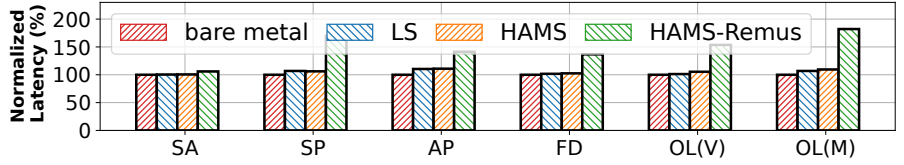


Fig. 9: Normalized latency of six AI serving software running on four systems.

Preliminary results. We implemented an prototype of **Obj. 2a & Obj. 2b** on AI serving micro-services and named it as Hams. Hams supports AI serving using Clipper [11] and Tensorflow Serving [12]. We evaluated Hams with 25 mature ML micro-services on PyTorch [138] and used these micro-service to build six practical ML serving software. We compared Hams with a bare metal system, a Hams-Remus system that follows Remus's primary-backup protocol, and an implementation of the most relevant replay-based approach LS [98]. Preliminary results (Fig. 9) shows that: Hams's end-to-end latency achieved 0.5% to 3.7% overhead compared with the bare metal system, and this overhead is comparable to LS's. Hams took 116.12ms ~ 254.19ms to recover one stateful model in all services, 155.1X ~ 1067.9X faster than LS.

2.3 A secure and high-performance micro-service partition system supporting both trusted TEE devices and untrusted accelerators.

This objective answers **RQ III**, by designing a secure and high-performance micro-service partition system that leverages both the strong security guarantees of trusted TEE devices (**Obj. 1 & Obj. 2**) and high performance untrusted accelerators. Same as micro-kernels, the partition should minimize state spills across the partitioned micro-services [122, 139] to minimize synchronization and integrity checks. To do so, our insight is that many micro-service contains linear parts that can be securely offload [85]. For each micro-services in the software (Fig. 10), the system first partitions each micro-service into a linear part and a non-linear part, and then transforms and offloads the linear part into an untrusted accelerator (e.g., GPU).

Obj. 3a: Morph-Then-Restore (MTR), a secure and high-performance partition protocol for running private micro-services on untrusted accelerators. The key observation of this objective is the general associativity property of linear operators in typical micro-services, as shown in Equation (1). This equation inspires a new morphing strategy: we can scale associative operators' parameters with a hidden scalar in TEE, partition these *morphed*, thus *confidentiality-preserved* operators to execute in GPU, and finally restore the accurate execution result with the scalar hidden in TEE without precision loss.

$$(\mu^{-1} * \mu) \cdot F(X) = \mu^{-1} \cdot F(\mu * X) \quad (1)$$

Based on this observation, we propose a new Morph-Then-Restore (MTR) protocol, which supports cooperative executions between trusted TEE devices (e.g., CPU and DPU TEEs) and untrusted accelerators (e.g., GPUs and NPUs). For each user input to the micro-service graph, MTR executes each operator (either in trusted TEE devices or untrusted accelerators) in order, transfers intermediate execution results (IR) across kernels when needed, and restores the final execution result with reciprocal coins in TEE (Fig. 11).

However, doing the morphing alone is insufficient to protect micro-service confidentiality, as a malicious attacker can observe IRs transmitted between trusted and untrusted kernels and deduce the hidden blinding coins with statistical analysis. Fig. 11 shows an illustrative example where the MTR protocol is integrated with IR morphing. If the IRs (Y_1 and Y_2) are not morphed with scalar μ_2 , an adversary who observes IRs will directly deduce the

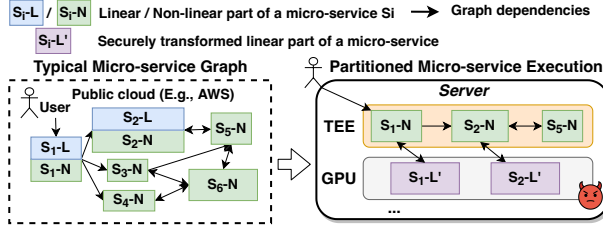


Fig. 10: Soter's secure micro-service partitioning.

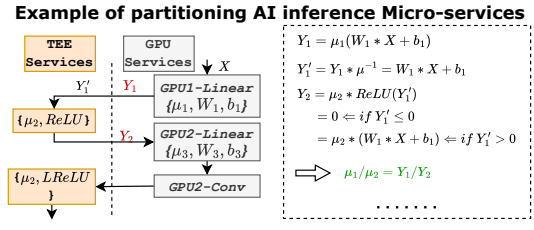


Fig. 11: Partition AI Serving micro-service.

plaintext value of the first Linear operator's blinding coin μ_1 by conducting a simple division (i.e., $\mu_1 = Y_1/Y_2$). The root cause of such coin leakage lies in the principle that the non-linear ReLU operator does not transform parameters with a positive value. Hence, confronted with that case, Soter's MTR protocol additionally morphs operators that are not partitioned to untrusted accelerators. By assigning a hidden blinding coin (i.e., μ_2) to the TEE operator ReLU, the adversary cannot deduce any blinding coins because the division of IRs can only get a scaled value with a hidden scale factor, thus protecting the confidentiality of partitioned micro-services.

Obj. 3b: Oblivious fingerprinting, an algorithm for efficiently and accurately detecting integrity breaches of partitioned micro-services on untrusted accelerators. However, as a partition-based system inevitably open access to arbitrary attacks outside the TEE (compared to a TEE-shielding system where all micro-services are running inside trusted TEE devices), it is urgent to develop a secure and fast mechanism to detect any integrity breaches of partitioned micro-services running on untrusted accelerators. Our key observation for this mechanism is that linear operators can use the linear combination property in the below Equation (2) to efficiently generate new fingerprints at runtime without re-running slow fingerprint generation in trusted TEE devices. Given that $F(X_1) = y_1, F(X_2) = y_2$, it always holds

$$F\left(\sum_{i=1}^n \mu_i * X_i\right) = \sum_{i=1}^n \mu_i * y_i \quad (2)$$

Concretely, the oblivious fingerprint technique takes two steps for detecting integrity breaches of partitioned micro-services. First, before running micro-services, several cornerstone fingerprints are precomputed and stored in a trusted TEE device. Second, during a partitioned service's execution, a random set of cornerstone fingerprints are selected. Then, by applying the linear combination property of Equation (2), we multiply each selected cornerstone fingerprint's input with a randomly generated coefficient with TEE's trustworthy random number generator, and add them all as the new input; for the fingerprint output, since the associativity property in Equation (2) implies that the corresponding output for a group of associated operators shares the same transformation, thus we can directly get the new fingerprint output by multiplying each selected cornerstone fingerprint's output with the same coefficient as the fingerprint's input. With these efficiently and securely generated fingerprints, Soter effectly detects integrity breaches of partitioned micro-services.

Preliminary result. We implemented a prototype of **Obj. 3a & Obj. 3b** for AI serving micro-services. We evaluated three notable baseline systems (MLCapsule [140], AegisDNN [141] and eNNclave [142]) that run complete all or partial ML operators of a micro-service within TEE. We use diverse ML micro-services (e.g., VGG19 [143], Alexnet [144], Resnet152 [145], Densenet121 [146], MLP and Transformer [147]) to build six prevalent ML serving software. Preliminary results (Fig. 12) show that Soter dramatically improved AI serving performance compared to MLCapsule (without GPU) and incurred moderate overhead than AegisDNN, while Soter uniquely guarantees micro-service confidentiality and integrity using techniques.

Obj. 3c: Unifying all aforementioned objectives for secure, reliable and fast micro-service graph executions. To ensure the integrity of micro-service execution (M1), this objective develops an onion integrity workflow, inspired by onion encryption [148]. The protocol conducts onion encryption of data along with the micro-service execution path (to form an OnionPath), so that the integrity of the execution can be attested by conducting onion decryption of the OnionPath. Leveraging this integrity protocols, we aim to integrate the secure OS architecture (**Obj. 1**) and the reliable execution techniques (**Obj. 2**) into modern micro-service software (e.g., Amazon ElasticCache [19], CloudTrail [19] and Google Cloud Platform [52]) with only a few code modification. We plan to evaluate the integrated systems on modern public clouds services such as AWS [19] and Huawei Cloud [110].

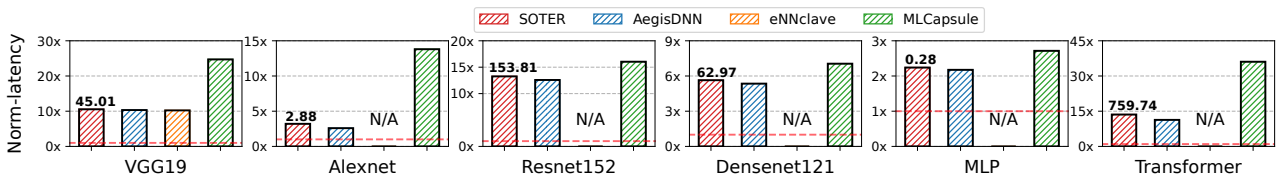


Fig. 12: Preliminary performance results of **Obj. 3a & 3b** on six AI serving software.

References

- [1] Shutian Luo, et al. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of the ACM Symposium on Cloud Computing*, 2021.
- [2] Anurag Khandelwal, et al. Jiffy: elastic far-memory for stateful serverless analytics. In *Proceedings of the Seventeenth European Conference on Computer Systems*, Rennes France, March 2022. ACM.
- [3] Haoran Zhang, et al. Fault-tolerant and Transactional Stateful Serverless Workflows.
- [4] Yanqi Zhang, et al. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, Virtual Event Germany, October 2021. ACM.
- [5] Dong Du, et al. Catalyzer: Sub-millisecond Startup for Serverless Computing with Initialization-less Booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, Lausanne Switzerland, March 2020. ACM.
- [6] Ana Klimovic, et al. Pocket: Elastic Ephemeral Storage for Serverless Analytics.
- [7] Xingda Wei, et al. No Provisioned Concurrency: Fast RDMA-codesigned Remote Fork for Serverless Computing, September 2022. arXiv:2203.10225 [cs].
- [8] Hao Zhou, et al. Overload Control for Scaling WeChat Microservices. In *Proceedings of the ACM Symposium on Cloud Computing*, October 2018. arXiv:1806.04075 [cs].
- [9] Yu Gan, et al. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019.
- [10] Alexandr Krylovskiy, et al. Designing a smart city internet of things platform with microservice architecture. In *2015 3rd international conference on future internet of things and cloud*. IEEE, 2015.
- [11] Daniel Crankshaw, et al. Clipper: A low-latency online prediction serving system. In *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, 2017.
- [12] Christopher Olston, et al. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv preprint arXiv:1712.06139*, 2017.
- [13] Han Vanholder. Efficient inference with tensorrt, 2016.
- [14] <https://memcached.org/>.
- [15] Apache storm. <http://storm.apache.org/>.
- [16] Nikita Lazarev, et al. Dagger: efficient and fast RPCs in cloud microservices with near-memory reconfigurable NICs. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Virtual USA, April 2021. ACM.
- [17] Hong Zhang, et al. Caerus: NIMBLE Task Scheduling for Serverless Analytics.
- [18] Alexandru Agache, et al. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX symposium on networked systems design and implementation (NSDI 20)*, 2020.
- [19] Amazon aws. <https://aws.amazon.com/>.
- [20] Azure. <https://azure.microsoft.com/>.
- [21] Uber micro-services. <https://eng.uber.com/microservice-architecture/>.
- [22] LinkedIn micro-services. <https://www.infoq.com/presentations/linkedin-microservices-urn/>.
- [23] Netflix micro-services. <https://www.nginx.com/blog/microservices-at-netflix-architectural-best-practices/>.
- [24] Akshitha Sriraman and Thomas F Wenisch. μ suite: a benchmark suite for microservices. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2018.
- [25] Haoran Qiu, Subho S Banerjee Saurabh, et al. Firm: An intelligent fine-grained resource management framework for slo-oriented microservices. *ile*.
- [26] Apache OpenWhisk. Open source serverless cloud platform. *Executes functions in response to events at any scale*, 2020.
- [27] Wei Zhang, et al. Towards qos-aware and resource-efficient gpu microservices based on spatial multitasking gpus in datacenters. *arXiv preprint arXiv:2005.02088*, 2020.
- [28] Sol Boucher, et al. Putting the "micro" back in microservice. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018.
- [29] Aws s3 outage. <https://aws.amazon.com/message/41926/>.
- [30] Aws lambda. <https://aws.amazon.com/lambda/>.
- [31] Aws fargate. <https://aws.amazon.com/fargate/>.
- [32] Azure confidential computing. <https://azure.microsoft.com/en-us/solutions/confidential-compute/>.
- [33] Vikram Sreekanti, et al. A fault-tolerance shim for serverless computing. In *Proceedings of the Fifteenth European Conference on Computer Systems*, Heraklion Greece, April 2020. ACM.
- [34] Nicolas Viennot, et al. Synapse: a microservices architecture for heterogeneous-database web applications. In *Proceedings of the Tenth European Conference on Computer Systems*, Bordeaux France, April 2015. ACM.
- [35] Nvidia multi-process service. https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf.
- [36] Aws micro-services. <https://awsosdi2021.splashthat.com/>.
- [37] Mingyu Li, et al. Confidential serverless made efficient with plug-in enclaves. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021.
- [38] Wei Zhang, et al. Astraea: towards QoS-aware and resource-efficient multi-stage GPU services. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Lausanne Switzerland, February 2022. ACM.
- [39] Dong Du, et al. Serverless computing on heterogeneous computers. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Lausanne Switzerland, February 2022. ACM.
- [40] Aws emr. <https://aws.amazon.com/emr/>.
- [41] Memched fpga acceleration. <https://www.xilinx.com/publications/events/developer-forum/2018-frankfurt/accelerating-memcached-on-cloud-fpgas.pdf>.
- [42] Intel fpga for storage. <https://www.intel.com/content/www/us/en/products/docs/storage/programmable/applications/storage.html>.
- [43] Nvidia dgx systems. <https://www.nvidia.com/en-us/data-center/dgx-systems/>.

- [44] Amazon ec2 f1 instances. <https://aws.amazon.com/ec2/instance-types/f1/>.
- [45] Dario Korolija, et al. Do OS abstractions make sense on FPGAs? In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, November 2020.
- [46] Jiacheng Ma, et al. A hypervisor for shared-memory fpga platforms. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020.
- [47] Nvidia open gpu kernel modules. <https://github.com/NVIDIA/open-gpu-kernel-modules>.
- [48] Andrew Baumann. Hardware is the new software. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*, 2017.
- [49] Haomiao Jiang, et al. Learning the image processing pipeline. *IEEE Transactions on Image Processing*, 2017.
- [50] Haoran Qiu, et al. FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [51] Amazon nitro enclaves. <https://aws.amazon.com/ec2/nitro/nitro-enclaves/>.
- [52] Google confidential computing. <https://cloud.google.com/confidential-computing>.
- [53] Intel. Software guard extensions programming reference. <https://software.intel.com/sites/default/files/329298-001.pdf>.
- [54] ARM. Security technology building a secure system using trustzone technology (white paper).
- [55] Dayeol Lee, et al. Keystone: An open framework for architecting trusted execution environments. In *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020.
- [56] Raad Bahmani, et al. CURE: A security architecture with customizable and resilient enclaves. In *30th USENIX Security Symposium (USENIX Security 21)*, 2021.
- [57] AMD. Amd secure encrypted virtualization (sev) development. <http://developer.amd.com/amd-secure-memory-encryption-sme-amd-secure-encrypted-virtualization-sev/>.
- [58] Erhu Feng, et al. Scalable memory protection in the PENGLAI enclave. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, July 2021.
- [59] Victor Costan, et al. Sanctum: Minimal hardware extensions for strong software isolation. In *USENIX Security Symposium*, 2016.
- [60] Top actions from gartner hype cycle for cloud security, 2020. <https://www.gartner.com/smarterwithgartner/top-actions-from-gartner-hype-cycle-for-cloud-security-2020>.
- [61] Jianyu Jiang, et al. Uranus: Simple, efficient sgx programming and its applications. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '20, New York, NY, USA, 2020. Association for Computing Machinery.
- [62] Jianyu Jiang, et al. Cronus: Fault-isolated, secure and high-performance heterogeneous computing for trusted execution environment. *IEEE/ACM International Symposium on Microarchitecture*, 2022.
- [63] Reto Acherhmann, et al. mmapx: uniform memory protection in a heterogeneous world. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2021.
- [64] Saurabh Gupta, et al. Failures in large scale systems: long-term measurement, analysis, and implications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017.
- [65] Guosai Wang, et al. What can we learn from four years of data center hardware failures? In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2017.
- [66] Shixiong Zhao, et al. Hams: High availability for distributed machine learning service graphs. In *Proceedings of the 50th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '20)*, 2020.
- [67] Jelena Mirkovic and Peter Reiher. A taxonomy of ddos attack and ddos defense mechanisms. *ACM SIGCOMM Computer Communication Review*, 2004.
- [68] Insu Jang, et al. Heterogeneous isolated execution for commodity gpus. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019.
- [69] Chia-Che Tsai, et al. Graphene-sgx: A practical library os for unmodified applications on sgx. In *2017 USENIX Annual Technical Conference (USENIX ATC)*, 2017.
- [70] Sergei Arnautov, et al. Scone: Secure linux containers with intel sgx. In *OSDI*, 2016.
- [71] Wencong Xiao, et al. Antman: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020.
- [72] Wencong Xiao, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [73] Sgx-lkl. <https://github.com/llds/sgx-lkl>.
- [74] Youren Shen, et al. Occlum: Secure and efficient multitasking inside a single enclave of intel sgx. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020.
- [75] Joshua Lind, et al. Glamdring: Automatic application partitioning for intel sgx. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, Santa Clara, CA, 2017.
- [76] Felix Schuster, et al. Vc3: Trustworthy data analytics in the cloud using sgx. In *Security and Privacy (SP), 2015 IEEE Symposium on*. IEEE, 2015.
- [77] Wenting Zheng, et al. Opaque: An oblivious and encrypted distributed analytics platform. In *NSDI*, 2017.
- [78] Shweta Shinde, et al. Panoply: Low-tcb linux applications with sgx enclaves. In *Proc. of the Annual Network and Distributed System Security Symp.(NDSS)*, 2017.
- [79] Stefan Brenner, et al. Securekeeper: Confidential zookeeper using intel sgx. In *Middleware*, 2016.
- [80] Fahad Shaon, et al. Sgx-bigmatrix: A practical encrypted data analytic framework with trusted processors. In *Proceedings of the 17th ACM conference on Computer and communications security (CCS '10)*, 2017.
- [81] Heejin Park, et al. Streambox-tz: secure stream analytics at the edge with trustzone. In *2019 USENIX Annual Technical Conference (USENIXATC 19)*, 2019.
- [82] Rishabh Poddar, et al. Visor: Privacy-preserving video analytics as a cloud service. In *29th USENIX Security Symposium (USENIX Security 20)*, 2020.

- [83] Zhongshu Gu, et al. Reaching data confidentiality and model accountability on the caltrain. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2019.
- [84] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. *arXiv preprint arXiv:1806.03287*, 2018.
- [85] Hanieh Hashemi, et al. Darknight: An accelerated framework for privacy and integrity preserving deep learning using trusted hardware. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [86] Tyler Hunt, et al. Chiron: Privacy-preserving machine learning as a service. *arXiv preprint arXiv:1803.05961*, 2018.
- [87] Alexander Schlögl and Rainer Böhme. ennclave: Offline inference with model confidentiality. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 2020.
- [88] Karan Grover, et al. Privado: Practical and secure dnn inference with enclaves. *arXiv preprint arXiv:1810.00602*, 2018.
- [89] Lucjan Hanzlik, et al. Mlcapsule: Guarded offline deployment of machine learning as a service. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [90] Fritz Alder, et al. S-faas: Trustworthy and accountable function-as-a-service using intel sgx. *arXiv preprint arXiv:1810.06080*, 2018.
- [91] Tien Tuan Anh Dinh, et al. M2r: Enabling stronger privacy in mapreduce computation. In *USENIX Security Symposium*, 2015.
- [92] Sinisa Matetic, et al. Rote: Rollback protection for trusted execution. *IACR Cryptology ePrint Archive*, 2017.
- [93] Christian Priebe, et al. Enclavedb: A secure database using sgx. In *Proceedings of the 2018 IEEE Symposium on Security and Privacy*. IEEE, 2018.
- [94] Fatima M Anwar, et al. Securing time in untrusted operating systems with timeseal. In *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2019.
- [95] Zhipeng Jia and Emmett Witchel. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, Virtual Event Germany, October 2021. ACM.
- [96] Brendan Cully, et al. Remus: High availability via asynchronous virtual machine replication. In *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation*, 2008.
- [97] Cheng Wang, et al. Plover: Fast, multi-core scalable virtual machine fault-tolerance. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*, 2018.
- [98] Stephanie Wang, et al. Lineage stash: fault tolerance off the critical path. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. ACM, 2019.
- [99] Ram Srivatsa Kannan, et al. Grand slam: Guaranteeing slas for jobs in microservices execution frameworks. In *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019.
- [100] Yaozu Dong, et al. Boosting gpu virtualization performance with hybrid shadow page tables. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, 2015.
- [101] Micah Dowty and Jeremy Sugerman. Gpu virtualization on vmware's hosted i/o architecture. *ACM SIGOPS Operating Systems Review*, 2009.
- [102] Saeid Mofrad, et al. Secdataview: a secure big data workflow management system for heterogeneous computing environments. In *Proceedings of The 35th Annual Computer Security Applications Conference (ACSAC'19)*, 2019.
- [103] Saeid Mofrad, et al. Securing big data scientific workflows via trusted heterogeneous environments. *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [104] Tianxiang Shen, et al. SOTER: Guarding black-box inference for general neural networks at the edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022.
- [105] Tianxiang Shen, et al. Daenet: making strong anonymity scale in a fully decentralized network. *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [106] Jianyu Jiang, et al. On the challenges of detecting side-channel attacks in sgx. In *25th International Symposium on Research in Attacks, Intrusions and Defenses*, 2022.
- [107] Tsz On Li, et al. Upa: An automated, accurate and efficient differentially private big-data mining system. In *Proceedings of the 50th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '20)*, 2020.
- [108] Jianyu Jiang, et al. Kakute: A precise, unified information flow analysis system for big-data security. In *Proceedings of the Annual Computer Security Applications Conference (ACSAC '17)*, 2017.
- [109] Shixiong Zhao, et al. Owl: Understanding and detecting concurrency attacks. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '18)*. IEEE, 2018.
- [110] Huawei tics. <https://www.huaweicloud.com/product/tics.html>.
- [111] Dingji Li, et al. Twinvisor: Hardware-isolated confidential virtual machines for arm. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP'21)*. ACM, 2021.
- [112] Stephen Checkoway and Hovav Shacham. *Iago attacks: Why the system call api is a bad untrusted rpc interface*, volume 41. ACM, 2013.
- [113] Wenhao Wang, et al. Leaky cauldron on the dark land: Understanding memory side-channel hazards in sgx. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2017.
- [114] Guoxing Chen, et al. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019.
- [115] Dayeol Lee, et al. An off-chip attack on hardware enclaves via the memory bus. In *29th USENIX Security Symposium (USENIX Security 20)*, 2020.
- [116] Adil Ahmad, et al. Obfuscuro: A commodity obfuscation engine on intel sgx. In *NDSS*, 2019.
- [117] Daniel Gruss, et al. Strong and efficient cache side-channel protection using hardware transactional memory. In *26th USENIX Security Symposium (USENIX Security 17)*, Vancouver, BC, August 2017. USENIX Association.
- [118] Olga Ohrimenko, et al. Oblivious multi-party machine learning on trusted processors. In *25th USENIX Security Symposium (USENIX Security 16)*, 2016.
- [119] Jianping Zhu, et al. Enabling rack-scale confidential computing using heterogeneous trusted execution environment. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020.

- [120] Tej Chajed, et al. Verifying concurrent, crash-safe systems with perennial. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.
- [121] Stavros Volos, et al. Graviton: Trusted execution environments on gpus. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018.
- [122] Gerwin Klein, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. ACM, 2009.
- [123] Andrew Baumann, et al. The multikernel: a new os architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, 2009.
- [124] Zeyu Mi, et al. Skybridge: Fast and secure inter-process communication for microkernels. In *Proceedings of the Fourteenth EuroSys Conference 2019*, 2019.
- [125] Trustzone security features. <https://developer.arm.com/architectures/architecture-security-features>.
- [126] Roberto Di Pietro, et al. Cuda leaks: a detailed hack for cuda and a (partial) fix. *ACM Transactions on Embedded Computing Systems (TECS)*, 2016.
- [127] Yuhong Zhong, et al. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.
- [128] Vasily A Sartakov, et al. CAP – VMs: Capability – Based isolation and sharing in the cloud. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.
- [129] Vta: Deep learning accelerator stack. <https://tvm.apache.org/docs/vta/index.html>.
- [130] <https://github.com/apache/tvm/tree/main/vta/tests/python/integration>.
- [131] Kaiming He, et al. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- [132] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [133] Yann LeCun, et al. The mnist database. <http://yann.lecun.com/exdb/mnist/>.
- [134] Alex Krizhevsky, et al. Learning multiple layers of features from tiny images. 2009.
- [135] Jia Deng, et al. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009.
- [136] Criu. <http://criu.org>, 2015.
- [137] Fritz Alder, et al. Aion: Enabling open systems through strong availability guarantees for enclaves, 2021.
- [138] Convolution Operator source code of PyTorch. <https://github.com/pytorch/pytorch/blob/master/aten/src/ATen/native/cudnn/Conv.cpp#L647>.
- [139] Kevin Boos, et al. A characterization of state spill in modern operating systems. In *Proceedings of the Twelfth European Conference on Computer Systems*, 2017.
- [140] Lucjan Hanzlik, et al. Mlcapsule: Guarded offline deployment of machine learning as a service. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [141] Yecheng Xiang, et al. Aegisdn: Dependable and timely execution of dnn tasks with sgx. In *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021.
- [142] Alexander Schlögl and Rainer Böhme. ennclave: Offline inference with model confidentiality. In *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 2020.
- [143] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [144] Alex Krizhevsky, et al. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 2012.
- [145] Kaiming He, et al. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- [146] Gao Huang, et al. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017.
- [147] Ashish Vaswani, et al. Attention is all you need. In *Advances in neural information processing systems*, 2017.
- [148] Raluca Ada Popa, et al. Cryptdb: protecting confidentiality with encrypted query processing. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. ACM, 2011.

Appendix (C): Non-text tables and figures

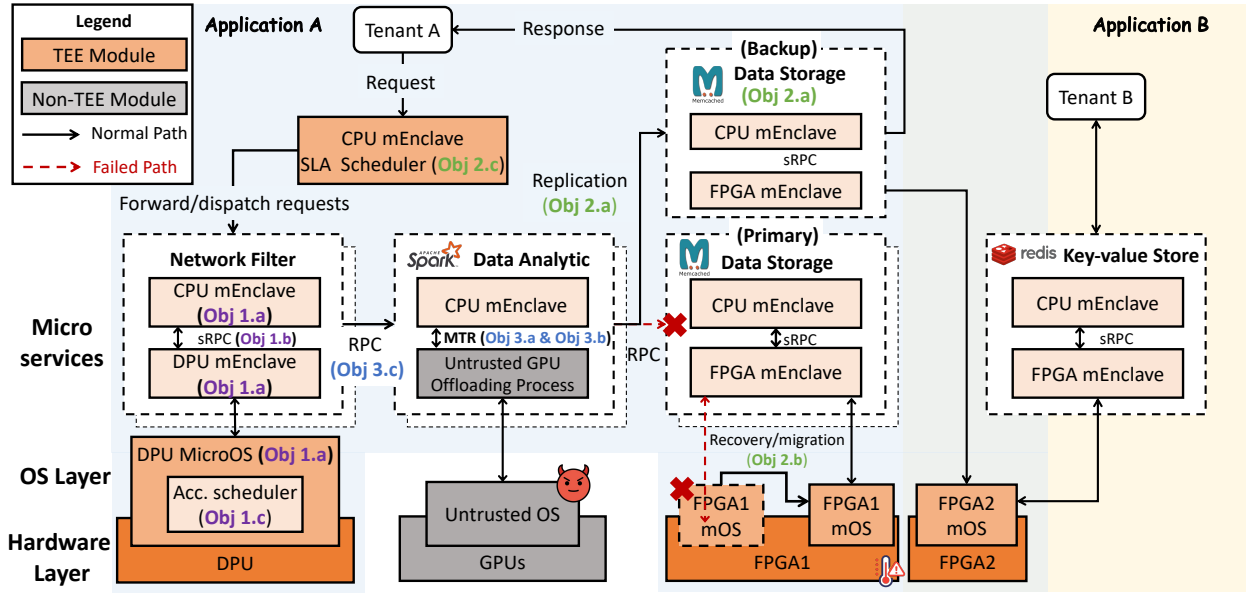


Figure 1: Complete micro-service execution workflow in the proposed system.

System	R1: Supporting Accelerators	R2: Spatial Sharing of Acc.	R3: Strong Isolation				RPC	
			FI:Enclave	FI:Kernel/Acc.	FI:Fault Recovery	R3.1: Fault Isolation (FI)	R3.2: Security Isolation (SI)	
						SI:Enclave	SI:Kernel/Acc.	
HARDWARE-BASED								
HETEE [?]	PCIe Acc.	$\times^1$	✓	✓	Slow ²	✓	✓	$\times$
CURE [?]	AXI Acc.	$\times^1$	✓	✓	Slow ²	✓	✓	$\times$
HIX [?]	GPU	✓	✓	N/A ³	Slow ²	✓	N/A ³	Slow ⁵
Graviton [?]	GPU	✓	✓	N/A ³	Fast ⁴	✓	N/A ³	$\times$
SGX-FPGA [?]	FPGA	✓	✓	N/A ³	Fast ⁴	✓	N/A ³	$\times$
SOFTWARE-BASED								
Panoply [?]	$\times$	N/A	✓	N/A ³	Fast	✓	N/A ³	Slow ⁵
EnclaveDB [?]	$\times$	N/A	✓	N/A ³	Fast	✓	N/A ³	Fast ⁶
TrustZone [?]	Generic	✓	✓	$\times$	Slow ²	✓	$\times$	Fast ⁷
Ji et. al. [?]	$\times$	N/A	✓	✓	Fast	✓	$\times$	Fast ⁷
Proposed system	Generic	✓	✓	✓	Fast	✓	✓	Fast

Remarks:

¹ Support temporal sharing but require cold-reboots of the accelerator when switching among enclaves.

² Require cold-reboot to clear accelerator states upon failures.

⁴ Require implementing failure recovery at hardware.

⁶ Defend against attacks at the application level.

³ FI and SI are not applicable when supporting one or no accelerators.

⁵ Uses encryption and acknowledgements for RPC, executing in lock-step.

⁷ Use monolithic OS for managing sharing memory.

Table 1: Comparison between the proposed system and related work.

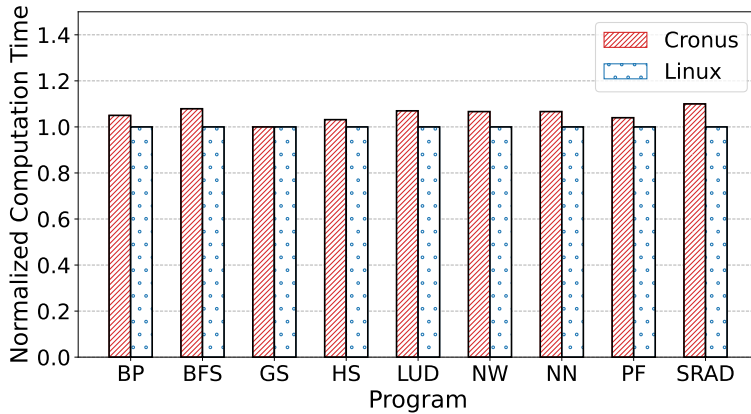


Figure 2: Figure 1

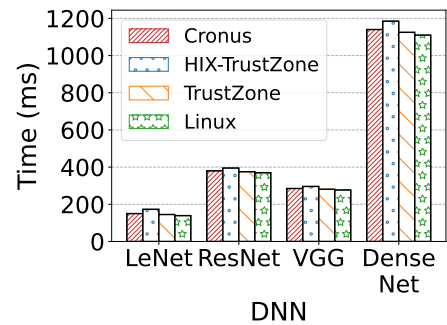


Figure 3: Figure 1

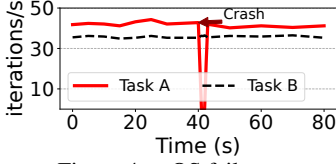


Figure 4: mOS failover.

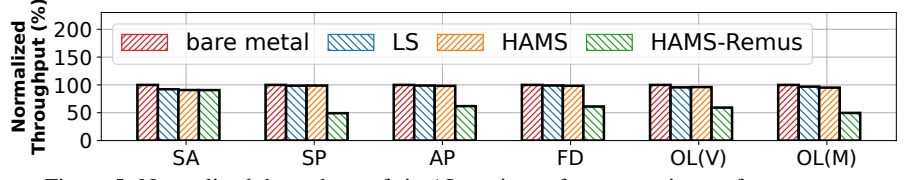
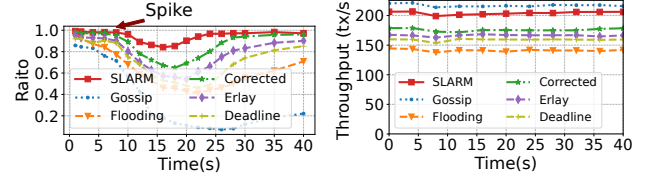


Figure 5: Normalized throughput of six AI serving software running on four systems.

	HAMS	HAMS-Remus	LS
SA	116.12ms	109.23ms	124.43s
SP	142.43ms	123.12ms	32.04s
AP	150.01ms	119.01ms	56.94s
FD	143.25ms	127.34ms	47.82s
OL(V)	254.19ms	315.42ms	62.10s
OL(M)	134.74ms	141.84ms	21.09s

Figure 6: Downtime time of Hams and related systems.



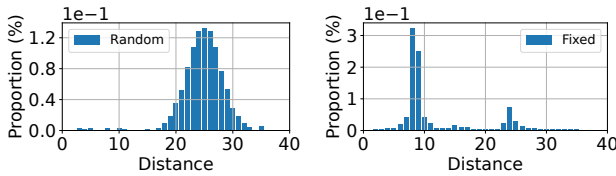
(a) SLA guarantees (b) Throughputs
Figure 7: SLA guarantee for SLA transactions and throughputs for all transactions on the online trading micro-services. At 0s, all systems are at peak throughputs; at 8s, a spike of 200 tx/s SLA transactions lasts for 5s.

Algorithm 1: MTR protocol at client c (offline).

```

1 Function partition( $O$ ) do
2   foreach operator  $op$  in  $O$  do
3      $op.partition \leftarrow \text{False}; op.index \leftarrow O.index(op);$ 
4     if  $op.index > W$  then
5       if  $0 < \text{normalize}(\text{sgx\_read\_rand}()) < \Theta$  then
6          $op.partition \leftarrow \text{True};$ 
7       if  $op$  is associative &  $op.partition$  then
8         foreach element  $e$  in  $\text{para}(op)$  do
9            $e \leftarrow \mu_{index} \times e;$  // Morph
10           $O_g.add(op);$  // GPU operators
11        else
12           $O_e.add(op);$  // TEE operators
13   $O_g.copyTo("cuda");$  // Partition to GPU

```



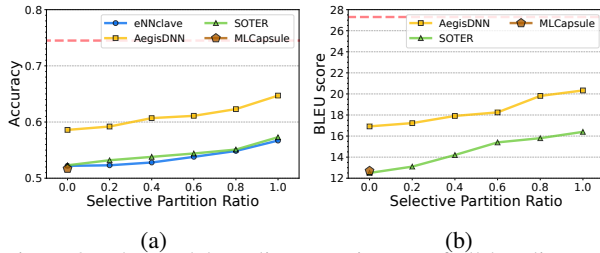
(c) w oblivious fingerprint. (d) w/o oblivious fingerprint.
Figure 8: The l_2 distance distribution of randomly sampled fingerprints with or without oblivious fingerprinting technique.

Algorithm 2: MTR protocol at client c (online).

```

1 Function secure_inference() do
2   forall  $I_1.$ 
3     every client input  $X$  do
4        $X.copyTo("enclave");$ 
5       foreach index  $i \in \text{normalize}(W)$  do
6          $X \leftarrow O_i(X);$  // Top-W Inference
7        $start \leftarrow \text{next } op \text{ in } O_g; end \leftarrow \text{next } op \text{ in } O_e;$ 
8       while  $start < |O|$  do
9          $I_2.$ 
10         $X.copyTo("cuda");$ 
11        foreach index  $i \in \text{range}(start, end)$  do
12           $X \leftarrow O_i(X);$  // GPU Inference
13         $I_3.$ 
14         $X.copyTo("enclave");$ 
15         $X \leftarrow \prod_{start}^{end-1} \mu_i^{-1} \times X;$  // Restore
16         $start \leftarrow end;$ 
17         $end \leftarrow \text{start from } O_{end} \text{ find the next } op \text{ in } O_g;$ 
18        foreach index  $i \in \text{range}(start, end - 1)$  do
19           $X \leftarrow O_i(X);$  // TEE Inference
20         $I_4.$ 
21         $X \leftarrow \prod_{start}^{end-1} \mu_i \times X;$  // Morph
22         $X \leftarrow \text{normalize}(X);$ 
23        return  $\text{Top}_1(X);$  // Final result

```



(a) (b)
Figure 9: The model stealing experiments of all baseline systems running (a) VGG19 and (b) Transformer. The red dotted line marks the ground-truth accuracy/BLEU score of VGG19 and Transformer respectively.

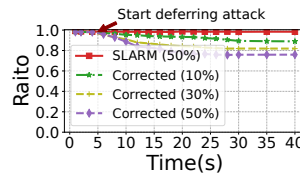


Figure 10: Portion of P2P nodes under deferring attack.

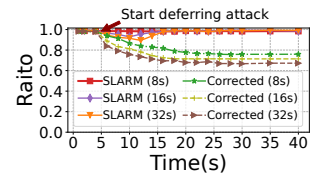


Figure 11: Extra latency added by the deferring attack.